

---

## CIVITAS

### Private Payroll Settlement Protocol

Zero-Knowledge Credentials · Solana Settlement · USDC Payroll

---

|               |                                           |
|---------------|-------------------------------------------|
| Version       | v2.0 (Solana Migration)                   |
| Date          | April 2026                                |
| Network       | Solana Devnet                             |
| Payroll Asset | USDC (SPL Token)                          |
| ZK Stack      | Noir · UltraHonk · BN254                  |
| Verifier      | Garaga<br>UltraKeccakZKHonkVerifier       |
| Auth          | Privy · Phantom / Solflare ·<br>Cartridge |
| Status        | Active — Solana Migration<br>Phase        |

## ABSTRACT

Civitas is a minimum-disclosure payroll settlement protocol that separates salary confidentiality from settlement finality. Employers compute payroll obligations off-chain, publishing only cryptographic Poseidon commitments and Merkle roots to a Solana payroll program. Employees redeem salary vouchers by submitting UltraHonk zero-knowledge proofs, generated locally from Noir circuits, proving credential knowledge without revealing their master nonce or salary amount. Spend uniqueness is enforced on-chain via an append-only nullifier registry. The protocol integrates Nillion nilDB confidential storage with secret-sharing semantics, optional AMD SEV-SNP trusted execution via nilCC, Privy and Cartridge-based wallet authentication, and USDC as the settlement

---

---

asset on Solana. This document provides the full technical specification: cryptographic definitions, Noir circuit semantics with modulo\_starknet bridging, program invariants, data model, lifecycle, and security analysis.

---

---

# 1. Introduction

Modern payroll systems, both traditional and Web3, impose a fundamental trade-off between operational transparency and individual privacy. A payroll processor must, by convention, possess complete salary data to execute disbursements, creating systemic risks: insider access to compensation tables, linkability between on-chain settlement transactions and employee identities, and total dependence on operator availability. For DAOs and decentralized organizations settling compensation on-chain, this problem is acute, every salary disbursement is permanently public, enabling competitors to reverse-engineer team burn rates, and exposing contributors to personal financial surveillance.

Civitas eliminates this trade-off by separating commitment to a payroll obligation from execution of that obligation. An employer publishes a cryptographic attestation encoding the full payroll commitment without disclosing any per-employee salary. An employee independently verifies inclusion and unilaterally redeems USDC without further employer interaction. The protocol is chain-native on Solana, leveraging its low transaction costs, fast finality, and the USDC SPL token standard, while keeping all privacy logic in the off-chain ZK layer.

## 1.1 Design Requirements

| Property                | Requirement                                                                                                                            |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Confidentiality         | Salary amounts, credential secrets, and voucher nonces remain encrypted at rest; only commitments are public.                          |
| Verifiability           | Any third party can verify a payroll run was committed to protocol without salary access via on-chain Merkle root and nullifier state. |
| Unforgeability          | Only the holder of a valid credential nonce can construct a valid UltraHonk withdrawal proof.                                          |
| Double-Spend Prevention | Each commitment redeemable at most once; enforced on-chain via append-only nullifier registry in the Solana program.                   |

---

|                              |                                                                                                     |
|------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Operator Independence</b> | After on-chain commitment, employees withdraw USDC without any further employer action required.    |
| <b>Hybrid Trust</b>          | Optional AMD SEV-SNP TEE (via nilCC) provides hardware-attested payroll generation confidentiality. |
| <b>Wallet Flexibility</b>    | Supports Privy, Phantom, Solflare, and Cartridge Controller wallet flows across user types.         |

Table 1. Civitas core design requirements.

## 2. System Architecture

Civitas comprises four cooperating layers, each with a distinct trust model and information boundary. The application layer orchestrates all workflows via a Next.js API and role-based dashboards. The cryptographic layer handles Noir ZK proof generation with the UltraHonk proving system. The confidential storage layer protects data at rest using Nillion nilDB secret-sharing. The settlement layer enforces all financial invariants unconditionally via the Solana payroll program and treasury vault, settling in USDC.

Figure 1 — Civitas Four-Layer Architecture (Solana)

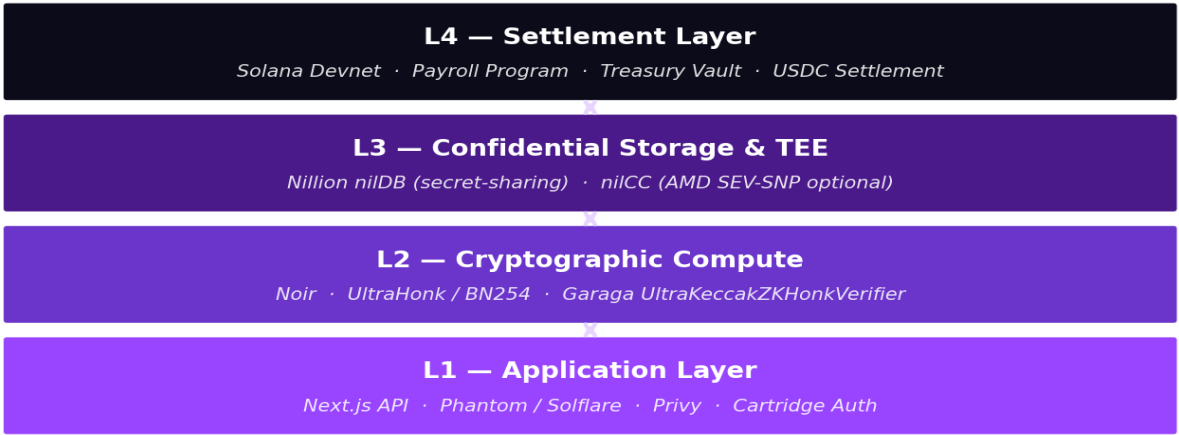


Figure 1. Civitas four-layer architecture on Solana. Arrows represent trust and data flow direction.

## 2.1 Payroll Information Flow

The swimlane below shows the end-to-end payroll flow across all system actors on Solana Devnet.

Figure 2 — Payroll Lifecycle Swimlane (Solana / USDC)

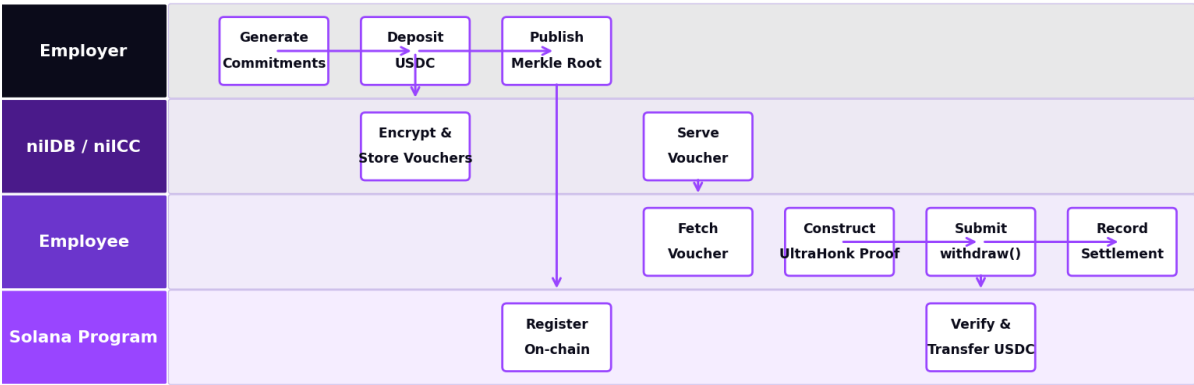


Figure 2. Payroll lifecycle swimlane: from commitment generation to on-chain USDC settlement.

## 2.2 Key Infrastructure Components

| Component              | Technology              | Role                                                                                                             |
|------------------------|-------------------------|------------------------------------------------------------------------------------------------------------------|
| Solana Payroll Program | Solana / Rust (planned) | Enforces financial invariants: nullifier registry, commitment gate, treasury vault solvency. Settlement in USDC. |
| Treasury Vault         | Solana PDA (planned)    | Program-derived address holding employer USDC deposits. Releases funds on valid proof submission.                |
| Noir ZK Circuit        | Noir / UltraHonk        | circuits/main.nr — encodes all 5 credential and Merkle                                                           |

---

|                        |                                  |                                                                                                            |
|------------------------|----------------------------------|------------------------------------------------------------------------------------------------------------|
|                        |                                  | constraints. Proofs generated client-side.                                                                 |
| <b>Garaga Verifier</b> | <b>UltraKeccakZKHonkVerifier</b> | Garaga-generated on-chain verifier contract. Validates UltraHonk proofs against the circuit VK.            |
| <b>Nillion nilDB</b>   | <b>Secret-sharing storage</b>    | Stores encrypted vouchers, employee tags, and payroll run metadata. No single node holds decryptable data. |
| <b>Nillion nilCC</b>   | <b>AMD SEV-SNP (optional)</b>    | TEE enclave for confidential payroll computation. Plaintext salaries never leave the hardware boundary.    |
| <b>Auth Layer</b>      | <b>Privy + Cartridge</b>         | Privy handles web2-style onboarding; Cartridge Controller handles native Solana wallet flows.              |

Table 2. Civitas infrastructure components. Items marked "planned" are in active migration.

---

### 3. Cryptographic Model

---

#### 3.1 Field Domains and Notation

Civitas operates across two prime fields with a bridging function.  $F_{BN254}$  is the BN254 scalar field used inside the Noir UltraHonk circuit for all ZK computations.  $F_p$  is the Solana-compatible 256-bit field used for all on-chain program state and nilDB storage. The `modulo_starknet` function in `main.nr` bridges these domains by reducing BN254 evaluation results into the target field range, ensuring that commitments, tags, and nullifiers computed in the circuit are directly comparable with values stored on-chain and in nilDB.

#### **Critical: modulo\_starknet Field Bridge**

All Poseidon hashes in the Noir circuit (C1–C3) are wrapped with `modulo_starknet(x)` — a function that reduces the BN254 Poseidon output into the target field range. This ensures that commitments registered on-chain, nullifiers in the program's nullifier map, and employee tags in nilDB are field-compatible with the values the circuit produces. Omitting this bridge causes silently incorrect verification with no runtime error.

#### 3.2 Core Cryptographic Relations

---

The diagram illustrates the commitment phase of the zk-SNARK process. It shows the following components and their relationships:

- $\eta$  (credential nonce)**: A black box containing the "master secret".
- $\tau$  (employee tag)**: A purple box containing the formula  $\text{Poseidon}(\eta) \bmod p$ . It is derived from  $\eta$  via a black arrow labeled  $\text{Poseidon} + \text{mod\_starknet}$ .
- $C$  (commitment)**: A purple box containing the formula  $\text{Poseidon}(\tau, a, e, v) \bmod p$ . It is derived from  $\tau$  and  $a, e, v$  via a black arrow labeled  $\text{Poseidon} + \text{mod } p$ .
- $N$  (nullifier)**: A red box containing the formula  $\text{Poseidon}(\eta, e, v) \bmod p$ . It is derived from  $\eta$  and  $v$  via a red arrow labeled  $\text{Poseidon} + \text{mod } p$ .
- $v$  (voucher nonce)**: A purple box containing the text "fresh random  $\in \text{BN254}$ ". It is derived from  $C$  via a black arrow.
- $a, e$  (amount, epoch)**: A teal box containing the text "employer inputs".
- olana Program Register**: A green box at the bottom, which is the source of the inputs  $a, e$ .

The three fundamental protocol relations, as implemented in `circuits/main.nr`:

|               |        |      |                                                                              |
|---------------|--------|------|------------------------------------------------------------------------------|
| Employee Tag: | $\tau$ | $:=$ | <code>modulo_starknet( Poseidon_BN254( <math>\eta</math> ) )</code>          |
| Commitment:   | $C$    | $:=$ | <code>modulo_starknet( Poseidon_BN254( <math>\tau, a, e, v</math> ) )</code> |
| Nullifier:    | $N$    | $:=$ | <code>modulo_starknet( Poseidon_BN254( <math>\eta, e, v</math> ) )</code>    |

| Symbol       | Domain  | Definition                                                                                                                              |
|--------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------|
| $\eta$ (eta) | F_BN254 | Master credential nonce — employee private secret. Generated client-side or in nilCC enclave. Never transmitted or stored in plaintext. |
| $\tau$ (tau) | F_p     | Employee tag — one-way derived from $\eta$ via Poseidon + modulo bridge. Shared with employer at onboarding only.                       |
| $a$          | u64     | Salary amount for epoch, encoded in USDC microunits (6 decimal places).                                                                 |



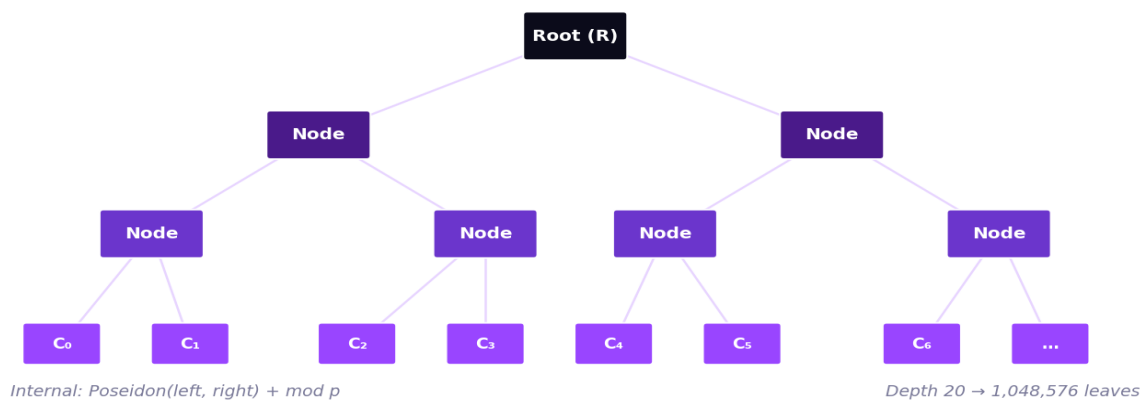
|               |     |                                                                                                                     |
|---------------|-----|---------------------------------------------------------------------------------------------------------------------|
| <b>e</b>      | u64 | Epoch identifier — monotonically increasing payroll period index.                                                   |
| <b>v (nu)</b> | F_p | Voucher nonce — freshly sampled random field element per employee per run. Prevents commitment reuse across epochs. |
| <b>C</b>      | F_p | Payroll commitment leaf — registered on-chain in the Solana payroll program.                                        |
| <b>N</b>      | F_p | Nullifier — published on-chain at withdrawal; marks commitment as spent.                                            |

Table 3. Protocol variable notation.

### 3.3 Merkle Commitment Tree

Commitments are inserted into a depth-20 binary Merkle tree with capacity  $2^{20} = 1,048,576$  leaves. Internal nodes are computed as  $\text{Poseidon}(\text{left}, \text{right})$  with the `modulo_starknet` bridge applied. The root  $R$  is published on-chain to the Solana program. For leaf index  $i$ , Noir's `std::merkle::check_membership` validates inclusion using an integer `path_index` (the leaf position as an integer) and a `merkle_path[20]` array of sibling hashes — replacing the bit-array encoding used in legacy Circom implementations.

Figure 4 — Depth-20 Poseidon Merkle Commitment Tree



---

Figure 4. Depth-20 Poseidon Merkle commitment tree. Noir uses integer `path_index` for inclusion proofs.

---

## 4. Zero-Knowledge Circuit (circuits/main.nr)

Civitas uses the UltraHonk proving system via the Noir language and the Barretenberg backend (bb). UltraHonk provides compact proof size, efficient verification, and critically — a manageable verifier contract footprint that deploys without payload rejection. The Garaga SDK generates the UltraKeccakZKHonkVerifier Cairo/Solana contract from the circuit's verification key. Proofs are generated entirely client-side in the browser using @noir-lang/backend\_barretenberg WASM.

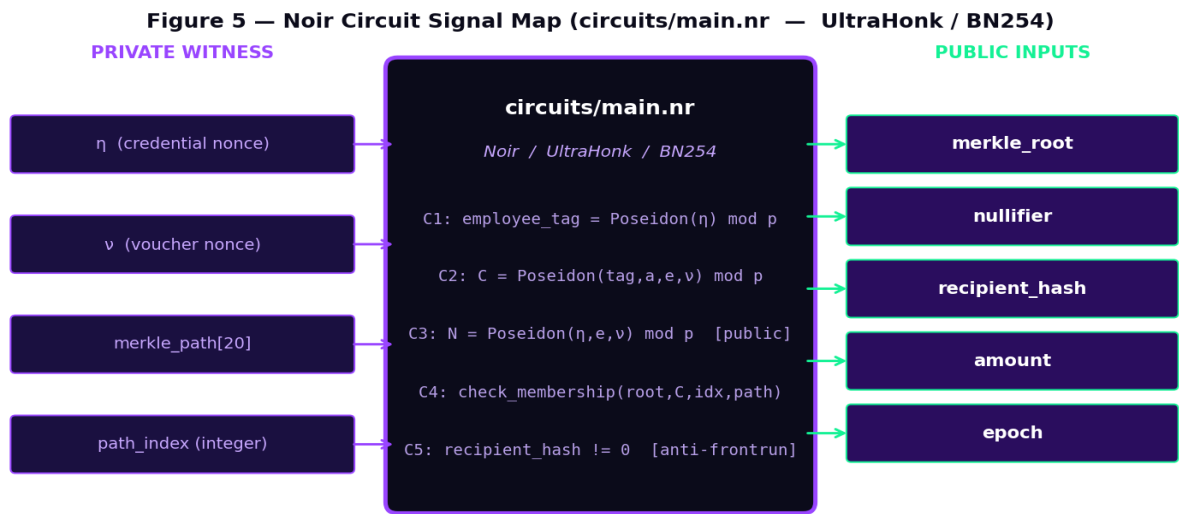


Figure 5. Noir circuit signal map — private witness (left), constraints (center), public inputs (right).

### 4.1 Circuit Constraints

|                                                                                                               |
|---------------------------------------------------------------------------------------------------------------|
| C1: <code>employee_tag = modulo_starknet( Poseidon_BN254( credential_nonce ) )</code>                         |
| C2: <code>commitment = modulo_starknet( Poseidon_BN254( employee_tag, amount, epoch, voucher_nonce ) )</code> |
| C3: <code>nullifier = modulo_starknet( Poseidon_BN254( credential_nonce, epoch, voucher_nonce ) )</code>      |

|                                                                                                    |
|----------------------------------------------------------------------------------------------------|
| [C3 equality asserted against public nullifier input]                                              |
| C4: <code>std::merkle::check_membership( merkle_root, commitment, path_index, merkle_path )</code> |
| [path_index is integer leaf position, not bit array – Noir native]                                 |
| C5: <code>assert( recipient_hash != 0 )</code> [binds proof to recipient – prevents front-running] |

## 4.2 Public and Private Signal Summary

| Signal                  | Visibility | Type      | Description                                                            |
|-------------------------|------------|-----------|------------------------------------------------------------------------|
| <b>credential_nonce</b> | Private    | Field     | Master secret $\eta$ — never leaves client device.                     |
| <b>voucher_nonce</b>    | Private    | Field     | Per-voucher randomness $v$ — from nilDB encrypted storage.             |
| <b>merkle_path[20]</b>  | Private    | Field[20] | Sibling hashes along the Merkle authentication path.                   |
| <b>path_index</b>       | Private    | Field     | Integer leaf position in the Merkle tree (replaces bit array).         |
| <b>merkle_root</b>      | Public     | Field     | Current Merkle root registered in the Solana program.                  |
| <b>nullifier</b>        | Public     | Field     | Spend token $N$ — checked against nullifier registry on-chain.         |
| <b>recipient_hash</b>   | Public     | Field     | Hash of recipient wallet address — prevents transaction front-running. |
| <b>amount</b>           | Public     | u64       | Salary amount in USDC microunits.                                      |
| <b>epoch</b>            | Public     | u64       | Payroll epoch identifier.                                              |

Table 4. Circuit signal table — private witness and public inputs.

---

### 4.3 Proof System Security Properties

| Property            | Guarantee                                                                                                                                             |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Completeness        | A prover knowing $(\eta, v, \text{path}, \text{path\_index})$ satisfying C1–C5 always produces an accepting proof.                                    |
| Knowledge Soundness | Any prover generating an accepting proof must know valid witnesses with overwhelming probability under the discrete log assumption.                   |
| Zero-Knowledge      | The proof reveals nothing about $(\eta, v, \text{path}, \text{path\_index})$ beyond the declared public inputs — $\eta$ stays computationally hidden. |

Table 5. UltraHonk security properties.

---

## 5. Solana Payroll Program

---

The Solana payroll program is the unconditional enforcement layer. It maintains the commitment registry, nullifier registry, treasury vault accounting, and USDC disbursement. All financial invariants hold regardless of off-chain behavior. The program interacts with the Garaga UltraKeccakZKHonkVerifier to validate ZK proofs before any fund release. The Solana program and treasury vault are currently in migration — the frontend layer is fully Solana-native, and program-side settlement is in active development.

### Migration Status

The frontend (frontend/lib/solana.ts, solana-wallet.tsx, civitas-provider.tsx) is fully migrated to Solana. SOLANA\_CLUSTER defaults to Devnet. SOLANA\_TREASURY\_VAULT and SOLANA\_PAYROLL\_PROGRAM define the program and vault addresses. USDC\_MINT\_ADDRESS defines the SPL token. Full on-chain program settlement is in active development. This section describes the target architecture.

### 5.1 Program State

| Variable               | Type           | Description                                                       |
|------------------------|----------------|-------------------------------------------------------------------|
| <b>authority</b>       | Pubkey         | Immutable employer authority — gates update_merkle_root().        |
| <b>merkle_root</b>     | [u8; 32]       | Current Merkle root; updated on each payroll commit.              |
| <b>treasury_vault</b>  | PDA (USDC ATA) | Program-derived USDC token account. Holds employer deposits.      |
| <b>total_deposited</b> | u64            | Cumulative USDC deposited into the treasury vault.                |
| <b>total_withdrawn</b> | u64            | Cumulative USDC withdrawn. vault_balance = deposited – withdrawn. |

---

---

|                         |                    |                                                                   |
|-------------------------|--------------------|-------------------------------------------------------------------|
| <b>nullifiers</b>       | Map<[u8;32], bool> | Append-only nullifier registry. Once set, never cleared.          |
| <b>commitments</b>      | Map<[u8;32], bool> | Commitment existence gate — only registered entries can withdraw. |
| <b>verifier_address</b> | Pubkey             | Address of the deployed Garaga UltraKeccakZKHonkVerifier.         |

Table 6. Solana payroll program state variables.

## 5.2 Program Instructions

|                                                                                                |
|------------------------------------------------------------------------------------------------|
| <code>deposit(amount):</code>                                                                  |
| <code>treasury_vault USDC balance += amount</code>                                             |
| <code>total_deposited += amount</code>                                                         |
|                                                                                                |
| <code>update_merkle_root(root, commitments[]):</code>                                          |
| <code>require(signer == authority)</code>                                                      |
| <code>register each commitment; merkle_root = root</code>                                      |
|                                                                                                |
| <code>withdraw(proof: UltraHonkProof, nullifier, commitment, amount, recipient):</code>        |
| <code>require( verifier.verify(proof) == true )      // ZK proof verified on-chain</code>      |
| <code>require( !nullifiers[nullifier] )                      // INV-1: no double-spend</code>  |
| <code>require( commitments[commitment] )                      // INV-2: registered only</code> |
| <code>require( vault_balance &gt;= amount )                      // INV-3: solvency</code>     |
| <code>nullifiers[nullifier] = true</code>                                                      |
| <code>transfer USDC from treasury_vault to recipient</code>                                    |

---

---

### 5.3 On-Chain Security Invariants

| Invariant                    | Mechanism                                                     | Guarantee                                     |
|------------------------------|---------------------------------------------------------------|-----------------------------------------------|
| INV-1: Nullifier Uniqueness  | nullifiers map is append-only; once true, never cleared.      | Second spend unconditionally rejected.        |
| INV-2: Commitment Gate       | withdraw() requires commitments[commitment] = true.           | Fabricated commitments cannot withdraw.       |
| INV-3: Vault Solvency        | Checks vault_balance >= amount per call.                      | Program cannot overdraft under any condition. |
| INV-4: Authority Exclusivity | update_merkle_root() gated exclusively to authority pubkey.   | Only employer can register new commitments.   |
| INV-5: ZK Proof Verification | withdraw() invokes Garaga verifier before any registry check. | Only valid UltraHonk proofs unlock funds.     |

Table 7. Solana payroll program security invariants.

---



## 6. Roles, Authentication, and Trust Boundaries

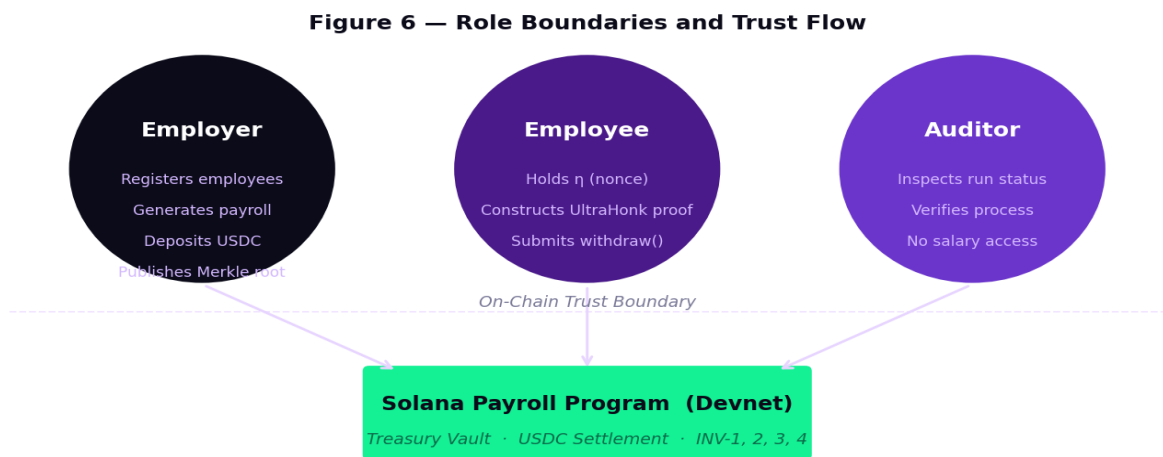


Figure 6. Role boundaries and trust flow to the Solana payroll program.

### 6.1 Roles and Capabilities

| Role     | Capabilities                                                                                                   | Information Access                                                                                                     |
|----------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Employer | Register employees; generate payroll; publish Merkle root; deposit USDC into treasury vault.                   | Knows $\tau$ (tags) and salaries during generation. Never learns $\eta$ . Authenticated via Privy or Phantom.          |
| Employee | Derive $\tau$ from $\eta$ ; receive vouchers from nilDB; construct UltraHonk proof locally; submit withdraw(). | Knows own $\eta$ , $a$ , $v$ for relevant epoch only. No access to other employees. Authenticated via Privy/Cartridge. |
| Auditor  | Inspect run status; verify process integrity; aggregate voucher state from nilDB.                              | Run-level metadata and commitment status only. No per-employee salary amounts.                                         |

Table 8. Role capabilities and information access boundaries.

---

## 6.2 Authentication Stack

| Provider             | User Type             | Flow                                                                                                                |
|----------------------|-----------------------|---------------------------------------------------------------------------------------------------------------------|
| Privy                | Employer,<br>Employee | Email/social login → embedded wallet. Lowers onboarding friction for web2-native users. Handles session management. |
| Phantom / Solflare   | Employer,<br>Employee | Native Solana wallet extension. Signs transactions directly. Standard for crypto-native users.                      |
| Cartridge Controller | Employee,<br>Auditor  | Solana-native session key controller. Enables gasless / batched interactions for employee withdrawal flows.         |
| ZK Credential Login  | Employee              | Employee authenticates using nonce-derived credential proof — no wallet signature required. Stateless auth.         |

Table 9. Authentication providers and use cases.

## 7. Data Model and Confidential Storage

Civitas maintains four logically isolated, per-company collections in Nillion nilDB. Sensitive fields use %allot secret-sharing semantics — no single nilDB node holds decryptable data. The merkle\_root and epoch fields are %allot encrypted to prevent pre-settlement monitoring.  $\eta$  (the master credential nonce) is never stored in any system — it lives exclusively on the employee's client device.

| Collection             | Key Fields                                              | Encryption Posture                                                  |
|------------------------|---------------------------------------------------------|---------------------------------------------------------------------|
| company_{id}_meta      | treasury_vault,<br>payroll_program,<br>authority_pubkey | Plaintext within nilDB access boundary; protected by access policy. |
| company_{id}_employees | $\tau$ (tag), PII fields,<br>USDC wallet address        | PII and salary data fully %allot encrypted. $\tau$ is               |

---

---

|                                  |                                    |                                                                                            |
|----------------------------------|------------------------------------|--------------------------------------------------------------------------------------------|
|                                  |                                    | plaintext index. $\eta$ is never stored.                                                   |
| <b>company_{id}_payroll_runs</b> | run_id, epoch, merkle_root, status | merkle_root and epoch are %allot encrypted — prevents pre-settlement monitoring.           |
| <b>company_{id}_vouchers</b>     | $\tau$ , amount, C, N, v, status   | All compensation and nonce fields fully %allot encrypted. Status is plaintext for queries. |

Table 10. nilDB collection schema and encryption posture.

7.1 Voucher Lifecycle

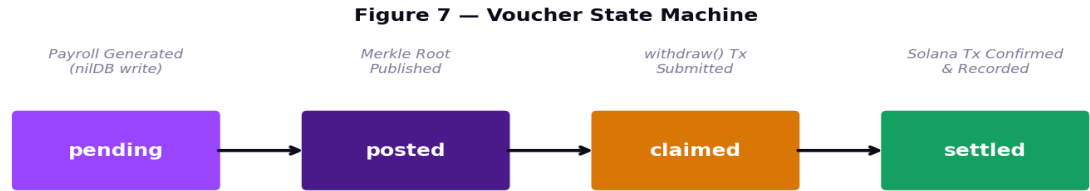


Figure 7. Voucher state machine — four states, three on-chain-triggered transitions.

---

---

## 8. Payroll Lifecycle

---

### 8.1 Employee Onboarding

| Mode              | Endpoint                            | Privacy Guarantee                                                                                                                             |
|-------------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Tag-Based         | /api/employer/employees/register    | Employee derives $\tau = \text{modulo\_starknet}(\text{Poseidon\_BN254}(\eta))$ client-side. Only $\tau$ is shared. $\eta$ never transmitted. |
| Credential Import | Employer-side bulk upload           | Employer imports pre-generated tag file. Suitable for bulk onboarding. $\eta$ generated per-employee client-side.                             |
| TEE Onboarding    | /api/employer/employees/onboard-tee | nilCC enclave generates $\eta$ inside AMD SEV-SNP hardware. Only $\tau$ returned to API. $\eta$ delivered out-of-band.                        |

Table 11. Supported employee onboarding modes.

### 8.2 Payroll Generation Paths

Path A — TEE (nilCC): Payroll manifest submitted to nilCC workload in AMD SEV-SNP enclave. Workload computes all commitments, voucher nonces, and Merkle root without exposing plaintext. API verifies attestation and stores encrypted results in nilDB.

Path B — Local Fallback: Server computes commitments using Poseidon + modulo\_starknet bridge directly. Provides storage confidentiality (nilDB) but not computation confidentiality. Produces identical on-chain artifacts to Path A.

### 8.3 Commit and Withdraw

POST /api/payroll/commit marks the run as committed and returns the calldata for the employer's wallet to invoke update\_merkle\_root(root, [C\_1...C\_n]) on the Solana program via treasury vault deposit. At withdrawal, the employee fetches their encrypted voucher from nilDB, decrypts the inputs, and generates a UltraHonk proof

---

locally using @noir-lang/backend\_barretenberg WASM. The proof is formatted via the Garaga SDK into Solana-compatible calldata, then submitted as a withdraw() instruction. POST /api/payroll/settle records the confirmed Solana transaction signature and updates the voucher status to settled.

9. Trusted Execution Environment (nilCC / AMD SEV-SNP)

AMD SEV-SNP (Secure Encrypted Virtualization — Secure Nested Paging) provides memory encryption for confidential virtual machines. All CVM memory is encrypted by the AMD Secure Processor using ephemeral keys inaccessible to the hypervisor. An attestation report includes a cryptographic MEASUREMENT over the workload code, allowing verification of the exact software executing in the enclave.

| Workload         | File                         | Function                                                                                                                                           |
|------------------|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Payroll Compute  | workload/run_compute.js      | Receives manifest (tags + salaries). Computes commitments and Merkle root inside enclave using modulo_starknet bridge. Plaintext never leaves CVM. |
| Blind Onboarding | workload/onboard_employee.js | Generates $\eta$ inside AMD SEV-SNP enclave; derives $\tau$ . Returns only $\tau$ to employer API. $\eta$ delivered out-of-band.                   |

Table 12. nilCC enclave workload specifications.

| Production Gap — Attestation Verification                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Current implementation verifies attestation structure and enclave type (SEV-SNP) only. Production-grade verification requires cryptographic validation of the AMD ARK→ASK→VCEK certificate chain, MEASUREMENT value pinning per workload, and freshness verification via nonce embedding. This is the top priority on the production roadmap. |

---

---

## 10. Security Analysis

### 10.1 Security Property Strength

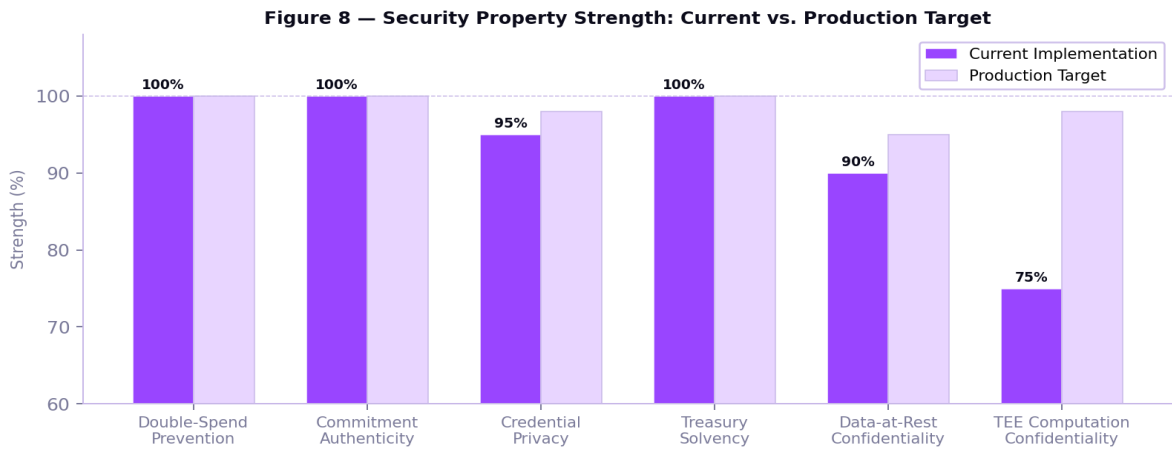


Figure 8. Security property strength — current implementation vs. production target.

### 10.2 Identified Risks and Mitigations

| Risk                                     | Severity        | Mitigation                                                                                |
|------------------------------------------|-----------------|-------------------------------------------------------------------------------------------|
| Structural-only attestation verification | Medium          | Implement AMD ARK→ASK→VCEK chain verification and MEASUREMENT value pinning per workload. |
| Solana program settlement in migration   | Medium (active) | Frontend is complete; program-side settlement is in active development. Devnet target.    |
| JWT secret policy — development defaults | High (pre-prod) | Replace with HSM-backed signing keys; implement rotation policy and audit logging.        |
| modulo_starknet bridge coverage          | Medium          | All C1–C3 constraints must consistently apply the bridge.                                 |

|                                                |            |                                                                                                   |
|------------------------------------------------|------------|---------------------------------------------------------------------------------------------------|
|                                                |            | Static validation tooling planned.                                                                |
| <b>Encrypted fields limit filtered queries</b> | <b>Low</b> | Add plaintext status index fields to voucher collection schema.                                   |
| <b>path_index encoding migration</b>           | <b>Low</b> | Legacy bit-array path encoding in any remaining test fixtures must be migrated to integer format. |

Table 13. Identified security risks and mitigations.

## 11. Performance and Scalability

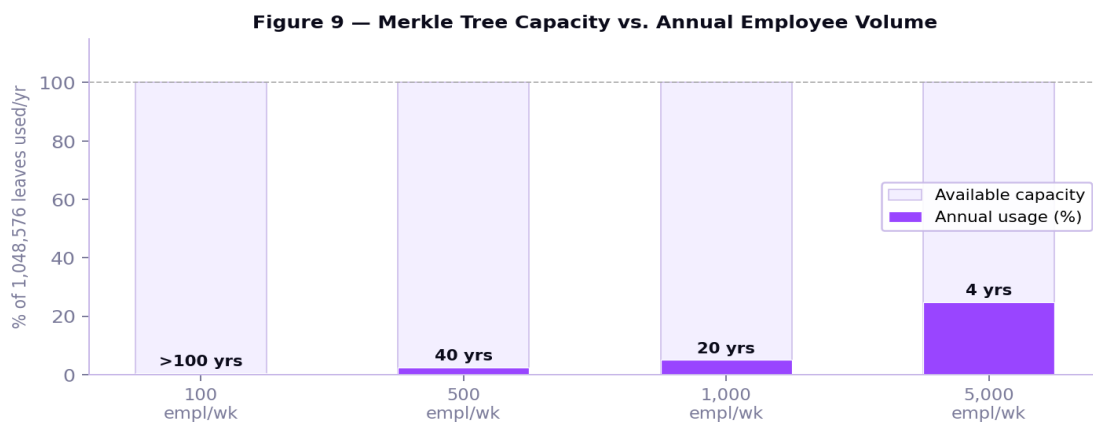


Figure 9. Merkle tree capacity: annual leaf usage vs. available capacity at different employee volumes.

| Operation                                | Complexity | Notes                                                                       |
|------------------------------------------|------------|-----------------------------------------------------------------------------|
| <b>update_merkle_root(n commitments)</b> | $O(n)$     | Linear in batch size. Efficient for bulk payroll posting to Solana program. |
| <b>withdraw()</b>                        | $O(1)$     | Constant: ZK verify + nullifier lookup + commitment check + vault transfer. |



---

|                                            |               |                                                                                          |
|--------------------------------------------|---------------|------------------------------------------------------------------------------------------|
| <b>deposit()</b>                           | O(1)          | Single SPL token transfer to treasury vault PDA.                                         |
| <b>UltraHonk proof generation (client)</b> | ~1–5 seconds  | Depth-20 circuit in WASM via @noir-lang/backend_barretenberg. Modern device performance. |
| <b>nilDB encrypted read/write</b>          | Network-bound | Additional latency proportional to nilDB cluster secret-sharing degree.                  |
| <b>Solana finality</b>                     | ~400ms        | Solana's fast finality benefits USDC withdrawal confirmation time significantly.         |

Table 14. Operational complexity and performance characteristics.

---

## 12. Production Hardening Roadmap

---

| # | Work Item                          | Priority | Description                                                                                               |
|---|------------------------------------|----------|-----------------------------------------------------------------------------------------------------------|
| 1 | AMD attestation-chain verification | Critical | Cryptographic AMD ARK→ASK→VCEK chain, MEASUREMENT pinning per workload, nonce freshness.                  |
| 2 | Complete Solana program deployment | Critical | Finalize on-chain program: nullifier registry, commitment gate, vault, USDC withdrawal. Deploy to Devnet. |
| 3 | JWT & key management hardening     | High     | HSM-backed signing keys, rotation policy, audit logging for all API authentication.                       |
| 4 | modulo_starknet static validation  | High     | Tooling to enforce bridge application at all API/circuit boundaries at build time.                        |
| 5 | Encrypted voucher indexing         | Medium   | Public status index fields for efficient lifecycle queries against %allot encrypted collections.          |
| 6 | path_index migration completeness  | Medium   | Audit all test fixtures and frontend proof generation for integer path_index compliance.                  |
| 7 | Auditor tooling expansion          | Low      | Deterministic run reproducibility verification without salary access for compliance workflows.            |
| 8 | Mainnet USDC production deployment | Low      | Graduate from Devnet to Solana Mainnet-Beta. Full USDC SPL token settlement in production.                |

Table 15. Production hardening roadmap.

---

## 13. Conclusion

---

---

---

Civitas demonstrates that salary confidentiality and settlement integrity are independently achievable and mutually compatible. By anchoring all financial obligations in a Solana payroll program with a treasury vault, enforcing spend uniqueness through an append-only nullifier registry, and protecting credential privacy through UltraHonk zero-knowledge proofs verified fully on-chain, Civitas eliminates the core failure modes of on-chain payroll systems without impractical cryptographic complexity.

The Noir ZK stack — with modulo\_starknet field bridging, integer Merkle path indexing, and client-side WASM proving — provides a clean, auditable cryptographic layer that avoids the deployment failures associated with legacy proving systems. The integration of Nillion nilDB secret-sharing and AMD SEV-SNP trusted execution provides defense-in-depth for data at rest and computation confidentiality. USDC on Solana provides sub-second settlement finality with minimal gas costs, making the protocol viable for real-world payroll volumes.

The frontend is fully Solana-native. On-chain program settlement is in active development targeting Solana Devnet. Civitas represents a technically rigorous foundation for private, verifiable, trustless payroll infrastructure — a concrete instantiation of the principle that financial privacy and unconditional enforcement are not in conflict.

---

---

## Appendix A: Technical Constants and API Reference

---

### A.1 Protocol Constants

| Constant              | Value                                                            |
|-----------------------|------------------------------------------------------------------|
| Merkle tree depth     | 20                                                               |
| Maximum leaf capacity | $2^{20} = 1,048,576$ commitments                                 |
| Settlement asset      | USDC (SPL Token, 6 decimal places)                               |
| Solana cluster        | Devnet (SOLANA_CLUSTER = devnet)                                 |
| Treasury vault        | SOLANA_TREASURY_VAULT (PDA — see frontend/lib/solana.ts)         |
| Payroll program       | SOLANA_PAYROLL_PROGRAM (see frontend/lib/solana.ts)              |
| USDC mint address     | USDC_MINT_ADDRESS (Devnet SPL USDC — see frontend/lib/solana.ts) |
| ZK proof system       | UltraHonk over BN254 (Noir / Barretenberg)                       |
| Verifier contract     | Garaga UltraKeccakZKHonkVerifier (generated from circuit VK)     |
| Auth providers        | Privy · Phantom · Solflare · Cartridge Controller                |
| Noir toolchain        | nargo 1.0.0-beta.3 · bb 0.85.0 · garaga latest                   |

Table A1. Protocol constants and deployment references.

### A.2 API Endpoint Reference

| Endpoint              | Method | Function                                                          |
|-----------------------|--------|-------------------------------------------------------------------|
| /api/payroll/generate | POST   | Initiate payroll generation — TEE (nilCC) or local fallback path. |

---

|                                            |      |                                                                               |
|--------------------------------------------|------|-------------------------------------------------------------------------------|
| <b>/api/payroll/commit</b>                 | POST | Mark run committed; return update_merkle_root() calldata for employer wallet. |
| <b>/api/payroll/settle</b>                 | POST | Record confirmed Solana transaction signature; update voucher to settled.     |
| <b>/api/payroll/attestation</b>            | GET  | Retrieve nilCC attestation object for a given payroll run.                    |
| <b>/api/employer/employees/register</b>    | POST | Tag-based or credential-import employee onboarding.                           |
| <b>/api/employer/employees/onboard-tee</b> | POST | TEE-based blind credential generation via nilCC enclave.                      |
| <b>/api/auth/login</b>                     | POST | Password / Privy-based login for employer and auditor accounts.               |
| <b>/api/auth/zk-login</b>                  | POST | ZK credential-derived login for employee accounts. Stateless.                 |

Table A2. Core API endpoint reference.

### A.3 Change Summary from Prior Architecture

| Category                | Removed                               | Replaced With                               |
|-------------------------|---------------------------------------|---------------------------------------------|
| <b>Blockchain</b>       | All prior L1/L2 settlement references | Solana · Solana Devnet                      |
| <b>Settlement Asset</b> | STRK token                            | USDC (SPL Token, 6 decimals)                |
| <b>Smart Contract</b>   | Cairo PayrollEscrow contract          | Solana Payroll Program (Rust, in migration) |

---

|                        |                                      |                                         |
|------------------------|--------------------------------------|-----------------------------------------|
| <b>ZK Language</b>     | Circom                               | Noir (circuits/main.nr)                 |
| <b>Proof System</b>    | Groth16                              | UltraHonk / BN254                       |
| <b>Verifier</b>        | Groth16 verifier (payload rejection) | Garaga<br>UltraKeccakZKHonkVerifier     |
| <b>Wallet Auth</b>     | StarkZap                             | Privy · Phantom / Solflare · Cartridge  |
| <b>Merkle Encoding</b> | path_indices[20] bit array (Circom)  | path_index integer (Noir std::merkle)   |
| <b>Field Bridge</b>    | Direct Poseidon_p (single field)     | Poseidon_BN254 + modulo_starknet bridge |

Table A3. Architecture migration change summary.